
World Models, JEPA et implémentation pratique avec Claude Code

Document unique consolidé : analyse scientifique, architecture cible, feuille de route et prompts opérationnels pour construire un world model latent moderne.

Version consolidée - Avril 2026

Préparé pour un usage recherche, prototypage et ingénierie ML.

Positionnement recommandé : démarrer par une V1 compacte, stable et modulaire, inspirée de JEPA / LeWorldModel, avant d'étendre vers la vidéo dense, la robotique réelle et les architectures hiérarchiques.

Objectif du document

Ce document rassemble en une seule version propre l'analyse scientifique sur les modèles de monde et les architectures JEPA, puis la transforme en feuille de route d'implémentation. La partie explicative a été reformulée pour devenir une introduction de référence, sans note de relecture ni ton préparatoire. Les prompts Claude Code sont intégrés à l'endroit où ils deviennent utiles : cadrage, architecture, données, modèle, entraînement, planification, évaluation et qualité logicielle.

Idée directrice
Un world model utile pour l'action ne doit pas nécessairement reconstruire chaque pixel. Il doit surtout apprendre un espace latent stable dans lequel les transitions, les effets des actions, la surprise physique et les objectifs de planification deviennent calculables.

Synthèse en une page

Axe	Décision recommandée	Raison
Paradigme	Prédiction latente inspirée JEPA	Évite de gaspiller la capacité du modèle sur les détails visuels peu utiles à l'action.
V1	World model compact depuis pixels	Permet un prototype entraînable localement avant les architectures vidéo massives.
Stabilité	Régularisation isotrope simplifiée	Réduit le risque de collapse et structure mieux l'espace latent.
Données	Dataset synthétique séquentiel	Rend le projet testable sans dépendre d'un environnement robotique réel.
Exploitation	Rollout latent + random shooting / CEM	Transforme le modèle appris en outil de planification.
Validation	Métriques, latents, surprise VOE	Mesure autre chose que la simple fidélité visuelle.

Sommaire

Partie I - Fondements scientifiques	World models, JEPA, variantes, LeJEPA, SIGReg, LeWorldModel, applications, évaluation physique et perspectives.
Partie II - Traduction en architecture logicielle	Objectifs d'ingénierie, arborescence, flux de données, composants logiciels et classes attendues.
Partie III - Prompts Claude Code intégrés par phase	Prompts prêts à coller, placés dans le déroulé logique de construction du projet.
Partie IV - Validation, usage et extensions	Conseils d'utilisation, résultat attendu de la V1, roadmap et bibliographie consolidée.

Partie I - Fondements scientifiques

1. Définir un modèle de monde

Un modèle de monde est un système qui apprend à représenter les lois de transition d'un environnement. À partir d'un état courant et, souvent, d'une action, il anticipe les états futurs. L'intérêt est de permettre à un agent de simuler des scénarios avant d'agir réellement : tester mentalement une trajectoire, prévoir une collision, estimer une récompense future ou détecter une violation physique.

La transition décrite dans le document source est essentielle : l'IA moderne ne cherche plus seulement à classer, générer ou compléter des données, mais à construire des représentations internes exploitables pour l'action. Les modèles de monde se situent donc au croisement de l'apprentissage auto-supervisé, de la robotique, de la vidéo, de la conduite autonome et du contrôle prédictif.

Concept	Définition pratique	Rôle dans le projet
World Model	Simulateur interne des transitions d'un environnement.	Base de l'imagination et de la planification.
Espace latent	Représentation compacte de l'état du monde.	Réduit la dimension et rend les rollouts rapides.
JEPA	Architecture qui prédit des embeddings plutôt que des pixels.	Paradigme principal de la V1.
SIGReg / isotropie	Contraintes statistiques sur la distribution des latents.	Stabilise l'apprentissage et limite le collapse.
VOE	Violation of Expectation : surprise face à une transition impossible.	Évaluation de la compréhension physique.

2. Taxonomie utile des world models

Pour structurer le champ, on peut classer les world models selon trois axes : la fonction visée, la manière de modéliser le temps, et la forme de la représentation spatiale. Cette taxonomie aide à situer une implémentation pratique : le projet visé ici n'est pas un générateur vidéo universel, mais un modèle latent compact destiné à prédire, évaluer et planifier.

Axe	Sous-catégorie	Description	Exemples
Fonctionnalité	Couplé à la décision	Optimisé pour le contrôle, la récompense et l'imagination de politiques.	DreamerV3, MuZero
Fonctionnalité	Usage général	Simulateur de dynamique pour prédictions futures diversifiées.	Sora, V-JEPA, Cosmos-Predict
Temps	Simulation séquentielle	Évolution de z_t via une transition récurrente ou auto-régressive.	RSSM, Dreamer
Temps	Différence globale	Prédiction de changements entre vues ou cadres sans récurrence stricte.	I-JEPA, V-JEPA
Espace	Vecteur latent global	État compact, efficace pour la planification rapide.	LeWM, I-JEPA
Espace	Géométrie explicite	Nuages de points, grilles d'occupation, champs neuronaux.	GaussianWorld, UniOcc

3. JEPA : prédire l'essentiel dans l'espace latent

L'architecture Joint-Embedding Predictive Architecture (JEPA), popularisée par Yann LeCun, propose un déplacement conceptuel majeur : au lieu de reconstruire directement les observations brutes, le modèle apprend à prédire une représentation latente cible à partir d'un contexte. La prédiction porte donc sur ce qui est utile et stable dans la scène, pas sur tous les détails visuels.

Dans un cadre classique, l'encodeur de contexte transforme la partie visible ou l'état courant en représentation. Un encodeur cible produit l'embedding de la partie à prédire ou de l'état futur. Un prédicteur apprend à rapprocher la représentation prédite de la représentation cible. Le risque principal est l'effondrement représentationnel : tous les exemples reçoivent un vecteur quasi constant. D'où l'intérêt de stratégies de stabilisation : EMA, stop-gradient, contraintes de variance, covariance, ou régularisation statistique.

Analogie
JEPA ressemble à la manière dont un humain anticipe une scène : il ne reconstruit pas chaque pixel mentalement, mais prédit les grandes structures utiles - déplacement, cause, obstacle, permanence, objectif.

4. De I-JEPA à V-JEPA

I-JEPA valide le principe sur l'image : prédire des blocs masqués dans l'espace latent, avec un ViT et une stratégie de masquage multi-blocs, sans dépendre principalement d'augmentations manuelles ni d'une perte pixel. V-JEPA transpose cette idée à la vidéo en prédisant des régions spatio-temporelles masquées. V-JEPA 2 et 2.1 renforcent la densité des représentations et l'utilité pour la compréhension vidéo et la robotique.

Modèle	Objectif	Intérêt pour notre projet
I-JEPA	Prédiction de blocs cibles masqués dans l'espace latent.	Fondation conceptuelle : apprendre sans reconstruction pixel dominante.
V-JEPA	Prédiction spatio-temporelle masquée dans les vidéos.	Extension naturelle vers la dynamique et le mouvement.
V-JEPA 2 / 2.1	Représentations denses, vidéo à grande échelle, usage robotique.	Inspiration future, mais trop ambitieux pour une V1 locale.
LeWorldModel	World model JEPA stable depuis pixels avec objectif latent compact.	Cible d'ingénierie la plus réaliste pour démarrer.

5. LeJEPA, SIGReg et la question de la géométrie latente

LeJEPA cherche à remplacer une partie des heuristiques de stabilisation par une justification statistique plus propre. L'idée centrale consiste à contraindre les embeddings à se rapprocher d'une distribution gaussienne isotrope : variance équilibrée dans toutes les directions, dimensions peu corrélées et structure favorable aux tâches aval.

SIGReg, ou Sketched Isotropic Gaussian Regularization, s'appuie sur l'idée que la distribution multivariée peut être caractérisée par des projections unidimensionnelles. En pratique, une implémentation complète peut être avancée ; pour une V1, une régularisation simplifiée combinant centrage de la moyenne, variance minimale et covariance hors diagonale constitue un excellent compromis d'ingénierie.

Choix de V1
Commencer par une régularisation isotrope simple est préférable à une reproduction fragile et incomplète de SIGReg. La conception doit toutefois garder une interface propre pour pouvoir remplacer cette régularisation plus tard.

6. LeWorldModel : cible réaliste pour l'implémentation

LeWorldModel est intéressant parce qu'il rapproche le paradigme JEPA d'un modèle de monde utilisable : observations brutes, embedding latent compact, prédiction de l'état suivant et régularisation de la distribution latente. Le document source souligne son orientation efficacité : peu de paramètres, faible coût de planification et structure adaptée à la robotique ou au contrôle latent.

Pour ce projet, LeWorldModel ne doit pas être cloné aveuglément. Il sert plutôt de boussole : une V1 légère, stable et lisible, entraînable sur dataset synthétique, puis extensible vers des environnements plus riches.

Principe	Traduction dans la V1
Observation -> latent compact	Encodeur CNN ou petit ViT configurable.
Transition latente conditionnée par action	LatentDynamicsModel $f(z_t, a_t) \rightarrow z_{t+1}$.
Loss principale non pixel	MSE ou cosine loss entre latent prédit et latent cible.
Régularisation	IsotropicLatentRegularizer avec mean/variance/covariance.
Planification	Random shooting puis CEM dans l'espace latent.

7. Applications : robotique, conduite autonome et planification

Les modèles de monde deviennent précieux lorsque l'action et la prévision sont liées. En robotique, l'agent peut simuler plusieurs trajectoires candidates avant d'exécuter la première action. En conduite autonome, un modèle latent peut aider à évaluer l'évolution probable d'une scène complexe. Dans les deux cas, la valeur du modèle dépend moins de la beauté visuelle d'un rollout que de la cohérence causale et physique de ses prédictions.

La boucle typique est : encoder l'observation courante, imaginer plusieurs séquences d'actions en latent, évaluer chaque trajectoire selon une fonction de coût, sélectionner la meilleure séquence, exécuter uniquement la première action, puis recommencer avec une nouvelle observation.

8. Évaluation : dépasser la fidélité pixel

Un modèle peut produire une vidéo visuellement plausible tout en violant des lois élémentaires : téléportation, disparition d'objet, collision incohérente, accélération impossible. Les métriques pixel ne suffisent donc pas. Une V1 utile doit mesurer la surprise latente : l'erreur de prédiction doit augmenter face à une violation physique, mais rester stable face à une variation superficielle comme un changement de couleur.

Le paradigme VOE, inspiré de la violation d'attente en psychologie du développement, donne un cadre simple : si le modèle a internalisé une régularité physique, une transition impossible doit générer une surprise plus forte qu'une transition normale.

9. Limites et perspectives

Les défis restent nombreux : dérive des embeddings sur les longs horizons, manque de cohérence temporelle, difficulté de représenter les contacts physiques, généralisation hors distribution, coût de l'évaluation et dépendance à la qualité des données. Les architectures hiérarchiques de type H-JEPA sont une piste prometteuse : niveaux bas pour les dynamiques rapides, niveaux hauts pour la planification à long terme.

L'objectif de ce document est donc de transformer une vision scientifique ambitieuse en progression logicielle crédible : d'abord un socle propre, ensuite des extensions vidéo, robotique, MPC avancé, benchmarks physiques et surprise multi-échelle.

Partie II - Traduction en architecture logicielle

10. Objectif d'ingénierie

La V1 doit fournir un laboratoire complet : génération de données, entraînement, évaluation, planification et analyse. Le projet doit rester assez simple pour être compris, testé et modifié, mais assez structuré pour devenir un vrai socle de recherche appliquée.

Principe de conception
Construire petit, propre et mesurable. Une V1 compacte qui apprend correctement sur un monde synthétique vaut mieux qu'une imitation incomplète d'un système vidéo géant.

11. Architecture cible du dépôt

```
/src
/world_model
/config
/data
/models
/training
/planning
/evaluation
/utls
/scripts
/tests
/configs
/docs
/artifacts
/checkpoints
/plots
/reports
```

Dossier	Responsabilité
config	Dataclasses ou Pydantic, chargement YAML, validation des hyperparamètres.
data	Dataset synthétique, générateur de séquences, collator, dataloaders, visualisation.
models	ObservationEncoder, ActionEncoder, Dynamics, heads, WorldModel, losses.
training	Trainer, checkpoints, métriques, logs, reprise d'entraînement.
planning	Random shooting, CEM, fonctions de coût, rollouts latents.
evaluation	Évaluation standard, surprise VOE, analyse des latents, rapports.
utls	Seed, device, logging, sérialisation, chemins, helpers.

12. Flux de données recommandé

```
observation_t -> ObservationEncoder -> z_t
action_t -> ActionEncoder -> a_t_emb
(z_t, a_t_emb) -> LatentDynamicsModel -> z_{t+1}_pred
observation_{t+1} -> TargetEncoderStrategy -> z_{t+1}_target
loss = prediction_loss(z_pred, z_target) + lambda * latent_regularization(z_target)
planner -> rollout_latent(world_model, z_t, candidate_actions) -> best_action
```

Cette structure sépare les responsabilités : l'encodeur apprend une représentation, la dynamique apprend la transition, la loss stabilise l'espace latent, le planner exploite les rollouts et l'évaluateur vérifie la cohérence du modèle.

13. Composants et signatures attendues

Classe	Signature ou méthode clé	Rôle
WorldModelConfig	from_yaml(path) -> WorldModelConfig	Centralise les hyperparamètres.
ObservationEncoder	forward(obs: Tensor) -> Tensor	Encode les images en z.
ActionEncoder	forward(action: Tensor) -> Tensor	Projette l'action dans l'espace utilisé par la dynamique.
LatentDynamicsModel	forward(z: Tensor, a: Tensor) -> Tensor	Prédit le prochain latent.
WorldModel	forward_step(obs_t, action_t, obs_tp1) -> dict	Orchestre encodeur, dynamique et cible.
WorldModelLoss	compute_loss(batch, outputs) -> dict	Combine prédiction et régularisation.
WorldModelTrainer	fit() -> TrainingResult	Gère entraînement, validation et checkpoints.
Planner	plan(z0, goal) -> PlanningResult	Cherche une séquence d'actions.
SurpriseEvaluator	evaluate(normal, violated) -> Report	Compare surprise normale et violation physique.

14. Feuille de route recommandée

Phase	Livrable	Prompt Claude Code associé
0	Cadrage, architecture, conventions	Prompts 1 et 2
1	Dataset synthétique séquentiel	Prompt 6
2	World model latent V1	Prompt 3
3	Refactor JEPA / LeWorldModel	Prompt 4
4	Régularisation isotrope	Prompt 5
5	Entraînement complet	Prompt 7
6	Planification latente	Prompt 8
7	Visualisation et analyse	Prompt 9
8	Surprise / VOE	Prompt 10
9	Tests, qualité, README	Prompts 11 et 12

Partie III - Prompts Claude Code intégrés par phase

Les prompts suivants sont placés dans l'ordre conseillé. L'idée n'est pas de demander tout le projet d'un coup, mais de contrôler l'itération : architecture, données, modèle, entraînement, exploitation, validation, puis documentation. Chaque prompt exige des fichiers complets, du typing, des docstrings et des tests afin d'éviter les réponses superficielles.

Mode d'emploi

Coller un prompt dans Claude Code, laisser l'agent modifier le dépôt, exécuter les tests, corriger, puis passer à la phase suivante. Le prompt tout-en-un doit rester un accélérateur, pas le mode par défaut.

Phase 0 - Cadrage et architecture

Cette phase force Claude Code à réfléchir avant d'écrire : structure du dépôt, responsabilités des classes, flux des données et conventions. Elle évite que le projet commence par un fichier unique difficile à maintenir.

Prompt 1 - Cadrage initial du projet

Tu es un ingénieur logiciel senior spécialisé en machine learning, PyTorch, world models, JEPA et architectures latentes pour l'IA incarnée.

Je veux que tu implémentes un projet Python complet, propre et structuré, pour un world model moderne inspiré de JEPA / LeWorldModel.

Objectif du projet :

- apprendre un état latent compact à partir d'observations image
- prédire l'état latent futur conditionné par l'action
- éviter la reconstruction pixel si possible
- utiliser une perte principale de prédiction latente
- prévoir une régularisation stable du latent inspirée d'une contrainte gaussienne/isotrope
- permettre ensuite la planification dans l'espace latent

Contraintes d'implémentation :

- Python 3.11
- PyTorch
- code modulaire, typé, documenté
- architecture claire orientée production/recherche
- tests unitaires minimum
- scripts CLI
- fichiers de configuration YAML
- entraînement reproductible
- support CPU et GPU
- journalisation des métriques
- sauvegarde/reprise de checkpoint
- dataset jouet généré si aucun dataset réel n'est disponible

Je veux que tu commences par :

1. analyser le besoin
2. proposer l'architecture complète du dépôt
3. créer tous les fichiers du projet
4. remplir chaque fichier avec du code complet, pas des stubs
5. ajouter docstrings, commentaires utiles, signatures complètes
6. générer un README détaillé
7. générer un script PowerShell create-project.ps1 qui crée toute l'arborescence du projet et les fichiers

Important :

- groupe les responsabilités par classe
- évite le code monolithique
- respecte SOLID autant que possible
- utilise dataclasses ou pydantic si pertinent pour la configuration
- n'oublie aucune méthode nécessaire
- explique les choix d'architecture dans le README

Avant d'écrire le code, donne-moi :

- la structure finale des répertoires
- la liste des classes principales
- le flux complet des données
- la liste des fichiers à générer

Prompt 2 - Architecture du dépôt

Implémente maintenant la structure complète du dépôt pour ce world model.

Je veux cette organisation logique :

```
/src
/world_model
/config
/data
/models
/training
/planning
/evaluation
/utils
/scripts
/tests
/configs
/docs
```

Exigences :

- séparer encodeur, modèle de transition latent, heads auxiliaires, losses, trainer, evaluator, planner
- créer des interfaces claires
- ajouter `__init__.py` partout
- ajouter typing complet
- ajouter docstrings de classes et méthodes
- créer un point d'entrée CLI pour :
 - train
 - evaluate
 - rollout
 - plan
 - visualize-latents

Je veux les fichiers complets, pas seulement la liste.

Phase 1 - Données synthétiques

Le dataset synthétique est la base de reproductibilité. Il permet d'entraîner et tester le modèle avant toute intégration avec un environnement réel.

Prompt 6 - Dataset synthétique séquentiel

Crée un module de données complet pour entraîner le world model sans dépendre immédiatement d'un dataset externe.

Je veux :

- un dataset synthétique de séquences visuelles
- une scène 2D simple avec objets mobiles
- actions simulées
- transitions déterministes ou légèrement bruitées
- support des séquences de longueur variable ou fixe

Implémente :

- `SyntheticWorldSequenceGenerator`
- `SyntheticWorldDataset`
- `SequenceBatchCollator`
- `DataModule` ou équivalent simple

Chaque échantillon doit contenir :

- observations : $T \times C \times H \times W$
- actions : $T \times \text{action_dim}$
- rewards optionnels
- dones optionnels
- métadonnées physiques simples si disponibles (position, vitesse)

Ajoute :

- script de visualisation d'un batch
- script pour sauvegarder des GIF de séquences
- tests de cohérence sur shapes et reproductibilité

Phase 2 - Modèle latent V1

On construit ici la première version exécutable : encodeur, dynamique latente, têtes auxiliaires et loss de prédiction.

Prompt 3 - V1 fonctionnelle du world model

Implémente maintenant une V1 entièrement fonctionnelle du world model.

Spécifications minimales :

- observations : images (C, H, W)
- actions : vecteur continu ou discret encodé
- encodeur convolutionnel ou petit ViT configurable
- projection latent_dim configurable
- modèle de transition $f(z_t, a_t) \rightarrow z_{t+1}$
- tête reward optionnelle : r_t
- tête done optionnelle : d_t
- perte principale : MSE entre z_{t+1} prédit et z_{t+1} cible encodé
- régularisation latent : variance/covariance/isotropie configurable
- support teacher target encoder optionnel mais désactivable
- entraînement stable avec mixed precision optionnelle

Je veux les classes suivantes avec code complet :

- WorldModelConfig
- ObservationEncoder
- ActionEncoder
- LatentDynamicsModel
- RewardHead
- DoneHead
- WorldModel
- WorldModelLoss
- WorldModelTrainer
- CheckpointManager

Ajoute :

- forward_step
- rollout_latent
- compute_loss
- training_step
- validation_step
- save / load

Le code doit être exécutable.

Phase 3 - Alignement JEPA / LeWorldModel

Cette phase remplace la V1 dans la philosophie JEPA : prédire des embeddings, stabiliser les latents, et préparer une régularisation plus rigoureuse.

Prompt 4 - Refactor JEPA / LeWorldModel

Refactorise le projet pour le rapprocher d'un paradigme JEPA / LeWorldModel.

Contraintes conceptuelles :

- éviter la reconstruction pixel comme objectif principal
- privilégier la prédiction dans l'espace latent
- séparer clairement :
 - encodeur d'observation
 - encodeur cible (si nécessaire)
 - prédicteur latent conditionné par l'action
- utiliser une perte de prédiction d'embedding futur
- ajouter une régularisation du latent inspirée d'une contrainte gaussienne/isotrope
- garder l'ensemble simple, stable et léger

Implémente :

- une classe JEPASyleLatentRegularizer
- une abstraction TargetEncoderStrategy
- la possibilité de choisir :
 - shared encoder
 - frozen target encoder
 - EMA target encoder
- une interface pour les pertes :
 - mse_latent_loss
 - cosine_latent_loss
 - isotropic_regularization_loss

Je veux aussi une note technique dans /docs expliquant :

- en quoi cette approche diffère d'un autoencoder reconstruction pixel
- pourquoi la prédiction latente peut être plus adaptée à la planification
- quels compromis sont faits dans cette implémentation par rapport à la littérature

Prompt 5 - Régularisation isotrope simplifiée

Ajoute une implémentation propre d'une régularisation de l'espace latent inspirée de l'idée d'isotropie gaussienne.

Objectif :

- encourager un espace latent stable
- limiter le collapse
- favoriser des dimensions informatives et bien réparties

Je veux :

- une classe `IsotropicLatentRegularizer`
- possibilité de combiner :
 - pénalité sur variance minimale par dimension
 - pénalité sur covariance hors diagonale
 - pénalité sur moyenne non centrée
- API simple :


```
compute(latents: Tensor) -> dict[str, Tensor]
```

Je veux que la loss retourne :

- `total_regularization_loss`
- `mean_penalty`
- `variance_penalty`
- `covariance_penalty`

Ajoute :

- tests unitaires
- configuration YAML
- logs TensorBoard ou CSV des composantes de perte
- commentaires expliquant comment remplacer plus tard cette version par une version plus proche d'une régularisation statistique de type SIGReg

Phase 4 - Entraînement

Une boucle d'entraînement propre est indispensable : checkpoints, reprise, logs, validation et métriques exportables.

Prompt 7 - Boucle d'entraînement complète

Implémente la boucle d'entraînement complète du world model.

Je veux :

- entraînement epoch-based
- validation
- early stopping optionnel
- sauvegarde du meilleur checkpoint
- reprise depuis checkpoint
- gradient clipping
- scheduler configurable
- mixed precision optionnelle
- seed globale
- barre de progression
- logs lisibles console + fichier

La classe `Trainer` doit :

- accepter un modèle, un optimizer, des dataloaders, une config
- gérer `train_one_epoch`
- `validate_one_epoch`
- `fit`
- `save_checkpoint`
- `load_checkpoint`
- `export_metrics_to_csv`

Ajoute aussi :

- `train.py`
- `evaluate.py`
- un exemple de config YAML complet

Phase 5 - Planification

Le world model devient utile quand il sert à imaginer des actions. On commence par random shooting, puis on ajoute CEM.

Prompt 8 - Planification latente

Ajoute un module de planification dans l'espace latent.

Objectif :

- utiliser le world model pour imaginer plusieurs séquences d'actions
- choisir la meilleure selon une fonction de coût

Implémente une V1 simple avec :

- random shooting planner
- CEM planner optionnel
- fonction de coût configurable
- horizon de planification configurable
- support batch de candidats

Je veux les classes :

- LatentCostFunction
- RandomShootingPlanner
- CEMPlanner
- PlanningResult

Méthodes attendues :

- evaluate_action_sequences
- imagine_trajectory
- select_best_plan
- plan

Ajoute un script :

- plan_demo.py

Ajoute une documentation expliquant :

- comment la planification utilise rollout_latent
- quelles limites il faut attendre d'un modèle latent imparfait

Phase 6 - Analyse des représentations

L'analyse des latents vérifie si le modèle encode vraiment des variables pertinentes : position, vitesse, objectif, régularité physique.

Prompt 9 - Visualisation des latents

Ajoute des outils d'analyse des représentations latentes.

Je veux :

- extraction des latents sur dataset train/val
- PCA
- t-SNE si disponible
- UMAP optionnel
- tracés simples matplotlib
- corrélation latents vs variables physiques du dataset synthétique

Crée :

- LatentAnalyzer
- scripts/visualize_latents.py
- scripts/plot_latent_correlations.py

Je veux des figures enregistrées automatiquement dans /artifacts/plots.

Ajoute des tests simples et un README de la section analyse.

Phase 7 - Surprise et cohérence physique

Le module de surprise teste la différence entre incohérence physique et variation superficielle.

Prompt 10 - Surprise / violation d'attente

Ajoute un module d'évaluation de surprise inspiré du paradigme violation of expectation.

Objectif :

- mesurer si le world model réagit davantage à des transitions physiquement incohérentes qu'à des variations superficielles

Implémente :

- génération de séquences normales
- génération de séquences avec violation simple (téléportation, changement impossible de vitesse, collision incohérente)
- score de surprise basé sur erreur de prédiction latente
- rapport comparatif normal vs violation

Crée :

- SurpriseEvaluator
- scripts/evaluate_surprise.py
- un rapport markdown automatique avec tableaux et figures

Je veux une implémentation simple, propre et extensible.

Phase 8 - Qualité et documentation

La dernière phase transforme le prototype en projet réutilisable : tests, linting, README et documentation d'extension.

Prompt 11 - Tests et qualité logicielle

Ajoute une vraie couche de qualité logicielle au projet.

Je veux :

- tests unitaires pytest
- tests d'intégration sur un mini run d'entraînement
- vérification des shapes
- vérification des checkpoints
- vérification du planner
- vérification de la reproductibilité avec seed fixe

Ajoute aussi :

- pyproject.toml
- configuration ruff
- configuration black
- configuration mypy
- pre-commit si pertinent

Le projet doit passer les tests de base localement.

Prompt 12 - README final

Rédige un README.md complet et professionnel pour ce projet.

Le README doit contenir :

- objectif du projet
- principes de world model latent
- inspiration JEPA / LeWorldModel
- architecture logicielle
- architecture du modèle
- format des données
- procédure d'installation
- commandes d'entraînement
- commandes d'évaluation
- commandes de planification
- commandes de visualisation
- description des losses
- description des hyperparamètres
- roadmap V1 / V2 / V3
- limites connues

Je veux aussi une section :

- "Différences entre cette implémentation et les papiers de recherche"
- "Comment étendre vers vidéo, robotique et planification MPC"

Prompt accélérateur - version tout-en-un

À utiliser seulement lorsque le dépôt est vide ou lorsque l'on veut forcer Claude Code à générer une première version complète. Pour un résultat plus contrôlé, préférer les prompts par phase.

Tu vas implémenter un projet Python complet de world model latent inspiré de JEPA / LeWorldModel.

Objectif :

construire un système qui encode des observations visuelles en espace latent, prédit l'état latent futur à partir de l'état latent courant et de l'action, apprend sans reconstruction pixel comme objectif principal, utilise une loss de prédiction latente et une régularisation du latent favorisant stabilité et isotropie, puis permet une planification simple dans l'espace latent.

Technos :

- Python 3.11
- PyTorch
- pytest
- YAML configs
- matplotlib
- support CPU/GPU

Je veux une implémentation complète, modulaire, propre et exécutable avec l'arborescence suivante :

```
/src/world_model/config
/src/world_model/data
/src/world_model/models
/src/world_model/training
/src/world_model/planning
/src/world_model/evaluation
/src/world_model/utils
/scripts
/tests
/configs
/docs
```

Exigences :

- code complet dans chaque fichier
- pas de stubs
- docstrings détaillés
- typing complet
- architecture par classes
- tests unitaires
- scripts CLI
- README complet
- script PowerShell create-project.ps1 qui crée tous les répertoires et fichiers

Fonctionnalités minimales :

1. Dataset synthétique de séquences visuelles avec actions
2. Encodeur d'observations
3. Encodeur d'actions
4. Modèle de dynamique latente
5. Tête reward optionnelle
6. Tête done optionnelle
7. Loss de prédiction latente
8. Régularisation isotrope du latent
9. Entraînement complet avec checkpoints
10. Évaluation
11. Planification par random shooting puis CEM
12. Visualisation des latents
13. Module de surprise / violation d'attente simple

Je veux que tu procèdes dans cet ordre :

- A. proposer l'architecture logicielle et ML
- B. créer tous les fichiers
- C. implémenter le code complet
- D. ajouter les tests
- E. ajouter les configs
- F. rédiger le README
- G. exécuter les tests si possible et corriger les erreurs

Important :

- garde une V1 simple mais propre
- privilégie la lisibilité et la stabilité
- fais des choix raisonnables d'hyperparamètres par défaut
- documente les compromis par rapport aux papiers académiques
- n'oublie aucune signature de méthode

Partie IV - Validation, usage et extensions

15. Conseils d'utilisation avec Claude Code

- Toujours demander l'architecture avant le code complet.
- Exiger des fichiers complets et refuser les stubs non implémentés.
- Lancer les tests après chaque phase, puis demander une correction ciblée.
- Garder une V1 compacte : images simples, latents courts, dynamique MLP ou GRU, régularisation lisible.
- Ne passer à la vidéo, à la robotique réelle ou à H-JEPa qu'après validation de la boucle complète.

16. Critères de réussite de la V1

Critère	Indicateur concret
Entraînement	La loss de prédiction latente diminue sur train et validation.
Stabilité	Les latents ne collapsent pas : variance non nulle et covariance maîtrisée.
Rollout	Le modèle prédit plusieurs pas sans explosion numérique immédiate.
Planification	Le planner choisit des actions meilleures qu'une politique aléatoire simple.
Surprise	Les séquences violées produisent une erreur latente supérieure aux séquences normales.
Qualité logicielle	pytest passe, checkpoints rechargeables, config YAML reproductible.

17. Roadmap après la V1

Version	Objectif	Extensions
V1	World model image synthétique	CNN, MLP/GRU, loss latente, régularisation isotrope, random shooting.
V2	Vidéo et séquences plus longues	Masquage spatio-temporel, meilleur target encoder, CEM robuste.
V3	Robotique ou environnement simulé	Gymnasium, MuJoCo/PyBullet, objectifs, récompenses et MPC.
V4	Hiérarchie temporelle	Latents multi-échelles, H-JEPa, surprise à plusieurs horizons.
V5	Évaluation physique avancée	Benchmarks VOE, collisions, permanence d'objet, conservation.

18. Conclusion

La stratégie la plus solide consiste à relier clairement théorie et ingénierie. Les world models et JEPa donnent le cadre : apprendre des représentations prédictives utiles à l'action. LeWorldModel donne une cible réaliste : un modèle latent compact, entraînable, stable et exploitable par planification. Les prompts Claude Code traduisent cette vision en étapes de construction concrètes.

Le résultat attendu n'est pas seulement un modèle PyTorch. C'est un socle expérimental complet : données, modèle, entraînement, analyse, surprise, planification, tests et documentation. À partir de là, il devient possible d'étendre progressivement vers des scénarios plus ambitieux sans perdre la maîtrise du système.

Bibliographie consolidée

Les références ci-dessous reprennent la base bibliographique du document scientifique fourni. Elles sont conservées comme socle de travail et peuvent être reformattées selon le style académique choisi : IEEE, APA, ACM ou numérotation interne.

#	Référence	URL
1	[2510.16732] A Comprehensive Survey on World Models for Embodied AI	https://arxiv.org/abs/2510.16732
2	A Comprehensive Survey on World Models for Embodied AI	https://arxiv.org/html/2510.16732v1
3	Understanding World or Predicting Future? A Comprehensive Survey of World Models	https://arxiv.org/html/2411.14499v3
4	From language models to world models - Linc / CNIL	https://linc.cnil.fr/en/article-12-language-models-world-models
5	World Models Reading List: The Papers You Actually Need in 2025	https://medium.com/@graison/world-models-reading-list-the-papers-you-actually-need-in-2025-882f02d758a9
6	Learning to Model the World: A Survey of World Models in Artificial Intelligence	https://www.preprints.org/manuscript/202603.0739
7	JEPA: Joint-Embedding Predictive Architectures	https://aegean.ai/book/world-models/jepa
8	JEPA Wiki - Hugging Face Space	https://huggingface.co/spaces/mishig/jepawiki
9	14 JEPA Milestones as a Map of AI Progress - Turing Post	https://www.turingpost.com/p/jepamap
10	BijEPA: Bi-directional Joint Embedding Predictive Architecture	https://arxiv.org/html/2603.00049v1
11	Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture	https://arxiv.org/abs/2301.08243
12	VICReg: Variance-Invariance-Covariance Regularization	https://iclr.cc/virtual/2022/poster/6481
13	LeWorldModel: Stable End-to-End Joint-Embedding Predictive Architecture from Pixels	https://arxiv.org/html/2603.19312v1
14	V-JEPA 2.1: Unlocking Dense Features in Video Self-Supervised Learning	https://arxiv.org/html/2603.14482v1
15	V-JEPA 2.1 - Emergent Mind	https://www.emergentmind.com/papers/2603.14482
16	Daily Papers - Hugging Face, V-JEPA 2	https://huggingface.co/papers?q=V-JEPA%202
17	LeJEPA: Provable and Scalable Self-Supervised Learning Without the Heuristics	https://arxiv.org/abs/2511.08544
18	LeJEPA: Scalable Self-Supervised Learning - Emergent Mind	https://www.emergentmind.com/topics/lejepa
19	Sketched Isotropic Gaussian Regularization (SIGReg)	https://www.emergentmind.com/topics/sketched-isotropic-gaussian-regularization-sigreg
20	AI 101: What is LeJEPA?	https://www.turingpost.com/p/lejepa

#	Référence	URL
21	Paper page - LeWorldModel - Hugging Face	https://huggingface.co/papers/2603.19312
22	LeWorldModel: 15 Million Parameters to Map the World	https://evoailabs.medium.com/leworldmodel-15-million-parameters-to-map-the-world-9a0b37581279
23	lucas-maes/le-wm: Official code base for LeWorldModel	https://github.com/lucas-maes/le-wm
24	Navigation World Models - ResearchGate	https://www.researchgate.net/publication/394584280_Navigation_World_Models
25	World4Drive: End-to-End Autonomous Driving via Intention-aware Physical Latent World Model	https://openaccess.thecvf.com/content/ICCV2025/papers/Zheng_World4Drive_End-to-End_Autonomous_Driving_via_Intention-aware_Physical_Latent_World_Model_ICCV_2025_paper.pdf
26	LeWorldModel review - Moonlight	https://www.themoonlight.io/de/review/leworldmodel-stable-end-to-end-joint-embedding-predictive-architecture-from-pixels
27	Planning with Imagination: Model Predictive Control in Latent Space	https://satyamcser.medium.com/planning-with-imagination-model-predictive-control-in-latent-space-1e51fa115482
28	From Generative Engines to Actionable Simulators	https://arxiv.org/html/2601.15533v1
29	T2VPhysBench: A First-Principles Benchmark	https://openreview.net/forum?id=PuxClrgBo2
30	WoW-World-Eval / Embodied World Model Evaluation	https://arxiv.org/html/2601.04137v1
31	Koustuv Sinha research works - ResearchGate	https://www.researchgate.net/scientific-contributions/Koustuv-Sinha-2163331023
32	Another Look at Looking Time: Surprise as Rational Statistical Inference	https://par.nsf.gov/servlets/purl/10118687
33	An fMRI investigation of expectation violation in magic tricks	https://www.frontiersin.org/journals/psychology/articles/10.3389/fpsyg.2015.00084/full
34	Surprise About Sensory Event Timing Drives Cortical Transients	https://www.jneurosci.org/content/38/35/7600
35	Revisiting Scaling Laws for Language Models	https://aclanthology.org/2025.acl-long.1163.pdf
36	LLM Scaling Laws Explained	https://www.buildfastwithai.com/blogs/llm-scaling-laws-explained
37	Video Generation Models as World Models	https://arxiv.org/html/2603.28489v1